通信原理课程设计

实验指导书

大连海事大学 通信工程教学系

2023-11

目录

第 1 章　引言 1
第 2 章　IEEE802.11a OFDM 基带发送端系统的搭建 1
1.1 实验目的 1
1.2 实验条件 1
1.3 实验要求 2
1.4 实验原理 2
1.5 参考设计 5
第 3 章　802.11a OFDM 基带接收端帧同步算法设计与验证 8
3.1 实验目的 8
3.2 实验条件 8
3.3 实验要求 8
3.4 实验原理 8
第 4 章　802.11a OFDM 基带接收端载波频率同步算法设计与验证 11
4.1 实验目的 11
4.2 实验条件 11
4.3 实验要求 11
4.4 实验原理 11
4.5 参考设计 12
第 5 章　802.11a OFDM 基带接收端信道估计算法设计与验证 14
5.1 实验目的 14
5.2 实验条件 14
5.3 实验要求 14
5.4 实验原理 14
5.5 参考设计 15
第 6 章　802.11a OFDM 基带接收端系统搭建 16
6.1 实验目的 16
6.2 实验条件 16
6.3 实验要求 16
6.4 实验原理 17
6.5 参考设计 17
第 7 章 软件无线电平台 ADALM-PLUTO 简介 20
7.1 PLUTO-SDR 的参数及功能特点 21
7.2 PLUTO-SDR 软件配置 22
7.3 IIO Oscilloscope 23
7.4 MATLAB 环境配置 27

第1章　引言

通信原理课程设计的主要任务是在 Matlab 开发环境下，使用.m 语言开展基于 PLUTO-SDR 软件无线电平台的 OFDM 系统设计实验。

IEEE802.11a 无线局域网标准作为典型的 OFDM 系统，是学习 OFDM 技术一个典型的应用，课程设计以这一标准为基础，对 OFDM 基带处理的算法、架构进行介绍和分析：将整个系统划分为多个基本通信模块，使用 Matlab 编程实现 IEEE802.11a 标准的基带发送端和接收端；通过 PLUTO-SDR 的射频端口收发，对典型的系统指标进行仿真分析和实际信道收发回环分析。

第2章　IEEE802.11a OFDM 基带发送端系统的搭建

自从 20 世纪 80 年代以来，OFDM 技术开始逐渐被应用于各种双向无线数据业务中，他们大都是利用了 OFDM 可以有效消除信号多径传播所造成的符号间干扰（ISI）这一特征。首先得到广泛应用的是在广播式音频和视频领域，如数字音频广播（DAB）、数字视频广播（DVB）。DAB 是在 AM 和 FM 等模拟广播基础上发展起来的，可以提供与 CD 相媲美的音质和其他新型数据业务。DAV 标准于 1995 年由欧洲电信标准化协会（ETSI）制定，是第一个使用 OFDM 的标准。接着在 1997 年基于 OFDM 的 DVB 标准也开始投入使用。1998 年 7 月，经过多次修改之后，IEEE802.11 标准组决定选择 OFDM 作为 WLAN（工作于 5GHz 频段）的物理层接入方案。

本节主要对 IEEE802.11a 基带系统进行搭建、仿真、实际收发测试实验，重点在如何将基带算法通过 Matlab 语言实现并通过硬件平台在实际的信道中验证，并且对比实际信道和仿真信道的差异。本节对 OFDM 和 IEEE802.11a 原理不做过多的描述，感兴趣的读者可以参考国内外已经出版的参考书。

1.1 实验目的

通过学习 IEEE802.11a 基带系统帧结构，使用 Matlab 编程语言实现 IEEE802.11a 基带发送端的设计。通过射频发送，结合频谱分析，深入了解 OFDM 系统 FFT 变换、保护间隔、带宽与频谱特征等关键技术的实际体现。

1.2 实验条件

PLUTO-SDR 平台一套，Matlab2013a(及以上)软件环境，WiFi 频段天线。

1.3 实验要求

使用 Matlab 搭建 IEEE802.11a 基带发送端，至少支持 6Mbps~54MBps 的一种模式。使用 Matlab 观察基带帧的时域波形和频域功率谱。通过 PLUTO-SDR 平台射频发送，使用频谱分析观测功率谱测量信号带宽等频谱特性。

1.4 实验原理

IEEE802.11a 基带物理层协议于 1998 年发布，读者可以参考“IEEE Std 802.11a-1999（R2003）”或中华人民共和国国家标准“GB 15629.1101-2006”。下面对物理层帧结构的主要参数做简要介绍，表 1-1 给出了 IEEE802.11a WLAN 系统的基本参数。

表格 1-1 IEEE802.11a 系统基本参数

参数	数值
基本采样频率	20MHz
数据符号时间	3.2us
循环前缀时间	0.8us
OFDM 符号时间 T	32.us+0.8us=4us
数据子载波数 N_d	48
导频子载波数 N_p	4
总子载波数 N	$N_d + N_p = 52$
子载波间隔 Δf	312.5KHz
总子载波占用带宽	$N \times \Delta f = 16.25MHz$
数据速率	6,9,12,24,36,48,54Mbps
调制方式	BPSK,QPSK,16QAM,64QAM
编码速率	1/2,2/3,3/4

在室内无线通道中，为了克服多径信道高达几百纳秒的时延扩展(一般按 200ns 计算)，IEEE802.11a WLAN 系统建议保护间隔(循环前缀，CP)的长度为最大时延长度的 3~4 倍，即建议为 800ns。而保护间隔（CP）的长度是整个标准设计中主要考虑的参数之一，它在一定程度上决定了其它标准参数的选择。为了提高数据传输率，降低 CP 在整个传送符号中的能量和时间比，标准设计 OFDM 符号时间为 4us（其中 OFDM 数据符号时间为 3.2us，CP 时间为 0.8us)。这样也就决定了子载波间隔的宽度为 312.5KHz(子载波间隔为 OFDM 数据符号时间的倒数)。

IEEE802.11a WLAN 系统中规定采用约束长度为 7，生成多项式为(133,171)g 的卷积编码方式，卷积编码后编码效率为 1/2，通过对编码后的信号进行凿孔（puncture）操作可分别产生编码效率为 2/3、3/4 的信号。同时为提高信号纠正突发性错误的能力，还应对编码后的信号根据情况采用不同交织深度的分组交织器进行交织，交织后的信号根据信道的不同条件使用不同的调制方式，如 BPSK、QPSK、16QAM、64QAM 进行映射。在 IEEE 802.11a WLAN 系统中只有 48 个子载波用于传输数据，4 个子载波(-21、-7、7、21)预留给导频信号(1、-1)使用，剩余的 12 个子载波没有使用。导频信号的目的是用来估计任何残余的相位误差，同时还可以用于信道估计。最后 OFDM 系统通过 IFFT／FFT 变换，加入 CP 和长、短训练序列实现 6Mbps～54Mbps 数据传输速率。

IEEE802.11a 对发送帧结构做了具体规定：每一帧由前导训练序列(Preamble)、Signal 域和数据域构成。其中前导训练序列由短训练序列和长训练序列，以及长短训练序列之间的保护间隔(GI2)构成。短训练序列由 10 个完全相同的短训练符号构成，长训练序列由 2 个完全相同的长训练符号构成。短训练符号和长训练符号的生成方式可参见标准文档。保护间隔(GI2)由长训练符号的第 33-64 位构成。前导主要完成自动增益控制(AGC)、分组同步估计、符号同步估计、频率同步估计和信道估计功能。要求在接收到实际数据进行正确译码前完成上述前导符号提供的功能。

设 M_S 为短训练序列中短训练符号数目， N_S 为每个短训练符号的采样点数目； M_L 为长训练序列中长训练符号数目， N_L 为每个长训练符号的采样点数目。IEEE802.11a WLAN系统规定短训练序列 $M_L=10$， $N_S=16$；长训练序列 $M_L=2$， $N_L=64$。GI2 间隔内采样点数为 $N_{GI2}=32$。每个数据段OFDM 符号采样点 $N_{SYS}=N_S+N_G=80$。其中 N_G 为OFDM数据符号保护间隔GI的采样点数， $N_G=16$，它是本符号最后16个采样点的重复。

前导首先由 10 个完全相同的短训练符号组成，每个短训练符号持续时间为 800ns，是每个 OFDM 数据符号持续时间的四分之一。可以利用序号为 4 倍的子载波来生成，在-26~+26 共 52 个子载波中，采用序号为{-24，-20，-16，-12，-8，-4，4，8，12，16，20，24}的子载波生成，详见标准文档。

根据OFDM载波频率同步算法，训练符号越短，能估计出的频偏范围越大，通常假设重复信号的持续时间为 T，则可以估计出频率偏移大小范围 $1/2T$,这是因为更高的频率偏移会导致相位超过 π 而无法估计。IEEE802.11a系统中，短训练训练序列采样间隔50ns（带宽20MHz），采样点数 $N=16$，则可以估计的最大频率误差为625KHz：

$$f_{\max}=\frac{1}{2\times16\times50\times10^{-9}}=625\text{kHz}$$

长训练序列的区别在于长度N=64，则可以估计的最大频率误差为156.25KHz。IEEE802.11g 系统规定的载波频率是2.4GHz，IEEE802.11a 载波频率5GHz 频段，假如振荡器最大频率误差为20ppm，则收发端振荡器最大误差和可达40ppm。最大可能频率误差值$\Delta f=40\times10^{-6}\times5\times10^{9}=200\text{kHz}$左右，可以看到最大可能频率误差值在短训练序列的符号估计范围之内，结合短训练序列的粗频偏估计和长训练序列的细频偏估计，可以较准确地估计出收发的载波频率误差，接收频率同步实验见后部分。另外采用10个短训练序列还可用来做自动增益控制(AGC)和帧同步检测。短训练序列后是长训练序列，长训练序列由两个长训练符号构成，每个长训练符号包含52个子载波，持续时间与一个OFDM 数据符号持续时间相同4us。长训练序列用于做精同步和精频偏校正。

训练序列后位Signal域，其中包括RATE和LENGTH部分，共24个bit 构成。Signal中的信息bit经过BPSK和1/2的卷积编码，可以得到6Mbps的信息传输速率。RATE 部分包括4个bit，用于传输OFDM 数据符号的系统信息。SIGANL域后是数据域，以上训练序列、SIGNAL和data构成一个完整的基带发送帧。调制方式、编码方式与数据速率的对应关系如表格1-2。

表格1-2 调制方式编码方式与速率对应关系

数据速率（Mbps）	调制方式	编码效率	编码比特/子载波	编码比特/OFDM 符号	数据比特/OFDM 符号
6	BPSK	1/2	1	48	24
9	BPSK	3/4	1	48	36
12	QPSK	1/2	2	96	48
18	QPSK	3/4	2	96	72
24	16QAM	1/2	4	192	96
36	16QAM	3/4	4	192	144
48	64QAM	2/3	6	288	192
54	64QAM	3/4	6	288	216

IEEE80.211a使用了52个子载波，k=0处的直流子载波不传输符号，由于IFFT 的算法基于2N点，故采用64点的IFFT。53个在频率分配时分别在编号低端和高端，有6个和5个空符号，即k=-32,…,-27,27,…31，这样就可以保证系统的子载波频谱集中，从而使系统占用的频谱带宽尽可能窄，以节约频谱资源，减小信道间干扰。所以，52个非零子信道映射到64点输入的IFFT当中应按照图1-1所指定的方式，把1~26映射到相同标号的IFFT输入端口；而子信道-26~-1被映射到38~63大的IFFT输入端口；其余的IFFT输入端口，即27~37输入空值。采用64点的IFFT，因为这系统的采样间隔为50ns，采样频率至少20MHz。

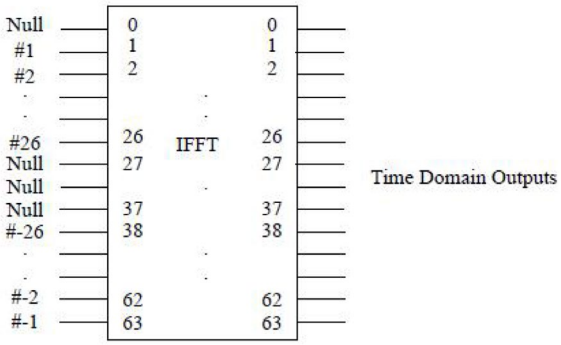


图1-1 IFFT 的输入和输出映射

1.5 参考设计

根据IEEE802.11a基带物理层标准，将各个功能模块化。首先设定PLUTO-SDR射频工作模式，然后读入系统参数、设定工作模式，之后分别产生训练序列、signal域与data域。图1-2是发送端的模块结构框图。

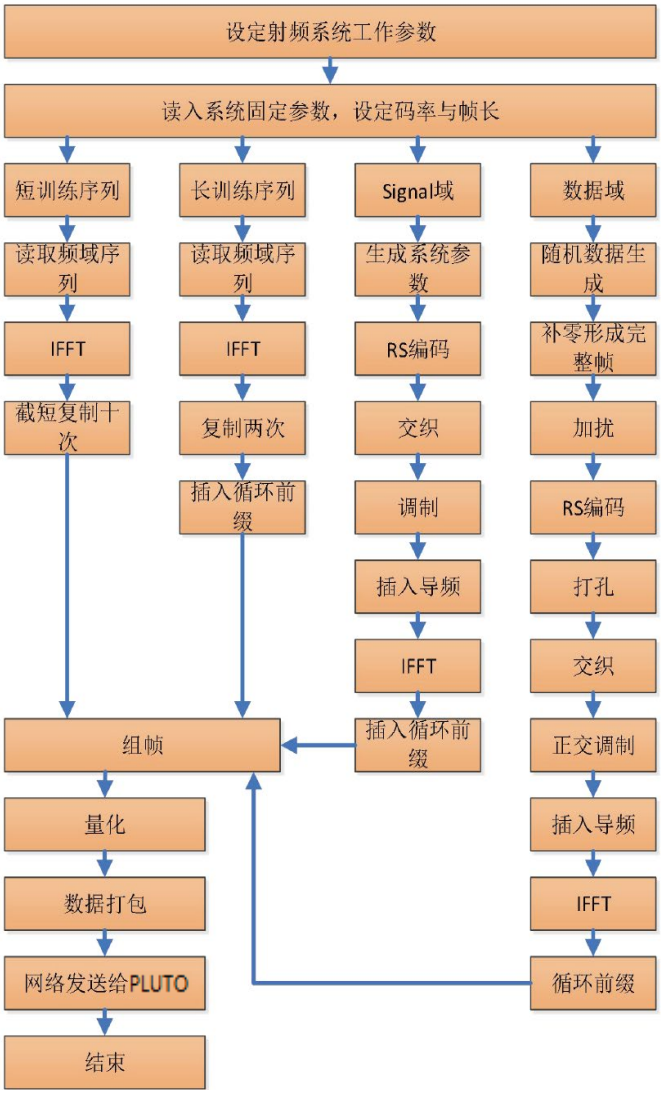


图1-2 IEEE802.11a 基带发送端结构图

1.5.1 射频系统参数设定

IEEE802.11a仅支持单天线,采样率设置40MHz；带宽设置20MHz；发送衰减如果通过天线传输建议设置10dB，如果通过电缆有线传输建议设置20dB。

1.5.2 PRBS 随机数列生成

PRBS 是Pseudo Random Binary Sequence 的缩写，即“伪随机二进制序列”的意思，也称为M序列。PRBS 码具有 “随机”特性，是因为在PRBS码流中，二进制数“0”和“1”是随机出现的，但是它又和真正意义上的随机码不同，这种“随机”特性只是局部的，即在周期内部，“0”和“1”是随机出现的(码流生成函数与初始码确定后，码流的顺序也是固定的)，但各个周期中的码流却是完全相同的，所以我们称其为“伪随机码”。PRBS 码的周期长度与其阶数有关，常用的阶数有7、9、11、15、20、23、31，也就是我们常说的PRBS7、PRBS9、PRBS11、PRBS15、PRBS20、PRBS23、PRBS31。

在对高速信号链路进行误码测试时，基本上都是利用PRBS 码流来模拟真实的线网码流环境，因为在线网中，所有的数据都是随机出现的，没有任何规律可言，而PRBS 码流在一定程度上具有这种“随机数据”特性，二进制“0”和“1”随机出现，其频谱特征与白噪声非常接近。PRBS 码流的阶数越高，其包含的码型就越丰富，就越接近真实的线网环境，测试的结果就越准确。

1.2.3 频域转换时域与插值

IEE802.11a的发送端使用64点IFFT。在做完子载波映射和插入导频后，每48个输入序列变成了53个输出序列，序号分别为-26到26共53个样值。然后将序号-26 到-1一共26个点放到64点IFFT的第38到63号位置，将序号1到26一共26个点放到64点IFFT的第1到26号位置上，这样就得到64点中的52点，然后再把其他位置补0，构成64点，之后就可以进行64点IFFT运算了，从而将频域信号转换到了时域信号。

IEEE802.11a是20MHz的采样率形成20MHz信号带宽。实际频谱两侧共留有11个零点，所以信号实际带宽53/64*20=16.5MHz。由于DAC的周期性混叠效应，通常在DAC与正交调制器之间加有低通滤波器，如本例中设置的滤波器带宽为20MHz。滤波器非理想，通常DAC采样率要至少高于带宽的2倍(40MHz以上)，可以将镜像推到更高的频段，这样可以实现较好的滤波效果。提高采样率的同时保证信号带宽，就必须采取插值滤波的方法。例如以下参考程序滤波器采用SRRC型，滚降系数为0.05，延时符号数为64，抽头数256。

虽然采样率提高，但是信号的带宽没有变化，但是SRRC滤波的效果随抽头系数的增加成正比，抽头系数越多需要占用的CPU资源更多，延时也更大。256点的抽头计算每个采样点需要做256次乘法和加法。

优化方案，插值的本质是将采样率提高 n 倍的同时，把有用频谱推挤到 $1/n$。具体方法是：对于即将要做IFFT变换的每64点数据，假设我们要求对数据进行 n 倍插值（这是为了跟硬件频率匹配），则我们在这64点中间插入 $(n-1)\times 64$ 个0，即将原来的64点分为前32点和后32点两部分，然后在这两部分中间插0，即“两边推”法。然后再将处理后的包括原来64点数据和插入的 $(n-1)\times 64$ 个0一共 $n\times 64$ 个点进行 $n\times 64$ 点的IFFT。两倍插值推挤频谱后(增加64个零值)做128点IFFT，这样达到了插值滤波的效果，还省掉了复杂的SRRC 滤波器，且频谱输出更加平滑。

1.2.4 量化与输出

按照IEE802.11a协议标准，生成归一化的数据帧，需要根据DAC的采样位宽将归一化数据量化。PLUTO-SDR射频前端采用ADI公司的AD9361全集成IC，集成的DAC位宽12bit，补码形式。因此，需要对归一化的I和Q数据的进行放大，一般是将I/Q数据乘以 2^{14}。

第3章　802.11a OFDM 基带接收端帧同步算法设计与验证

OFDM 基带的接收端不是简单的发送端的逆运算。由于通过DAC数模变换、射频的调制变频、信道、射频解调变频、ADC 采样，以上过程可以称为广义的信道，带来了信号的非线性畸变。接收系统如果想恢复出原始发送码流根据功能分成两大部分，信道补偿和比特逆运算。信道补偿主要是利用发送端的帧结构特性（比如训练序列、导频、循环前缀等）进行帧同步（寻找信号帧的起始点）、载波频偏补偿（消除收发系统载波的偏差）、信道估计与均衡（消除多径效应与信道衰落）、解星座映射等；比特逆运算是指，解交织、信道译码、解扰等，基本是发送端的逆处理。接收系统的框图如图3-1，由图可知，接收端的处理算法要比发送端复杂得多。

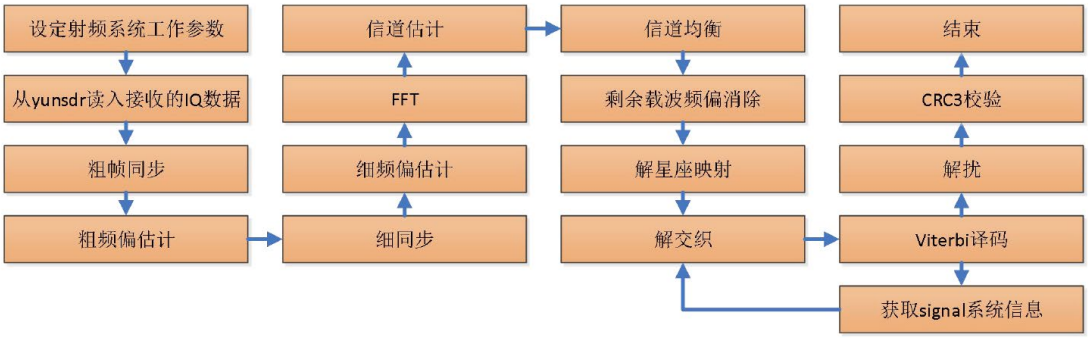


图3-1 IEEE802.11a 基带接收处理框图

3.1 实验目的

学习帧同步算法原理，通过软件仿真与实际PLUTO_SDR信道接收信号分别对同步算法进行验证。

3.2 实验条件

PLUTO_SDR 平台一套，Matlab 软件环境，天线或射频电缆回环。

3.3 实验要求

使用Matlab编写粗同步与细同步模块，通过软件仿真与实际信道验证。通过Matlab程序验证同步算法。

3.4 实验原理

IEEE802.11a系统是信道应答脉冲响应在数据突发期间,没有发生显著变化的假定前提下设计的。这样的假定在发送数据分组的极短时间内可以证明是合理的，通常是几毫秒，因为

发射机和接收机在大多数应用中互相间移动是较小的。该假设下大部分WLAN 接收机的同步可以在前导期间完成，并且在数据分组期间不需要改变。需要考虑的同步有三种，即：定时同步、频率同步和采样时间同步。本讲对定时同步进行介绍。

定时同步是估计出OFDM符号的起始位置，它主要包括两个部分：分组同步（粗同步）和符号同步（细同步）。如果符号后FFT 的起始位置在循环前缀（CP）的长度内，那么子载波间的正交性依然能够保持，在这种情况下，符号同步偏差可以看做是信道引起的相位旋转，这一旋转可以由信道估计求出并通过信道均衡消除。如果符号同步的偏差超过了保护间隔，就会引入载波间干扰（ICI）。导致无法正确计算出FFT窗口位置，误码率成倍提高。因此，OFDM 系统对定时同步的要求相对宽松，但是在多径信道环境中，为了获得最佳的系统性能，需要确定最佳的定时估计。尽管定时估计的起点可以在保护间隔内任意选取，但是容易得知，任何定时估计起点的变化，都会增加OFDM 系统对时延扩展的敏感程度，因此系统所能容忍的时延扩展会低于其设计值。为了尽量减小这种负面影响，需要尽量减小定时同步的估计误差。

系统定时估计算法可以分为两步完成，第一步通过分组检测进行粗同步（分组同步估计），第二步采用符号同步进行细同步（符号同步估计）。这两部同步估计都采用前导序列来完成。分组检测是指判断出数据分组到达的准确位置，这是一种粗同步，同时也是随后进行频率同步和符号同步的基础，剩下的同步过程都依赖于分组检测完成的优劣。在分组检测中以下声明代表是否出现了分组：

H_0 ：没有出现分组； H_1：出现分组

实际检测中通常是看判决变量 m_n 是否超出了阈值 T_h。分组检测的情况如下：

$H_0 : m_m < T_h$ 没有出现分组； $H_1 : m_n \geq T_h$ 出现分组。

通常的方法是采用双滑动分组检测技术，基本原理如下：双滑动窗口的分组检测算法计算了两个连续滑动窗口的能量，分别是后面窗口的自相关能量和前后窗口的互相关能量。因为短训练序列的特性是10个连续相同的符号进行重复，所以当窗口移动到信号的开始时互相关的能量和自相关的能量应该相差很小。信号运算流程图如图3-2：

其中：L 等于滑动窗口长度，通常采用短训练序列长度，即 $L=16$。图中有两个滑动窗口，分别用于计算 C_n 和 P_n，C_n 为接收信号和接收信号延时的互相关系数，叫做延时相关；延时 D 等于前导起始的周期，在IEEE 802.11a WLAN 系统中 $D=16$，也就短训练符号的周期长度； P_n 是互相关系数窗口期间接收信号的能量，此值用于判决统计的归一化。其中

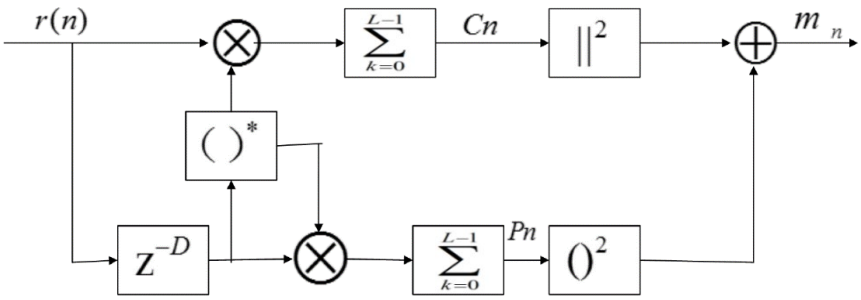


图3-2粗同步分组检测流图

$$C_n = \sum_{k=0}^{L-1} r(n+k)r^*(n+k+D)$$

$$P_n = \sum_{k=0}^{L-1} r(n+k+D)r^*(n+k+D) = \sum_{k=0}^{L-1} \left| r(n+k+D) \right|^2$$

$$m_n = \frac{\left| C_n \right|^2}{P_n^2}$$

根据IEEE 802.11la WLAN 系统建议，判决变量 $m_n = 0.75$ 时的第一个取样标识n作为分组同步估计的起始点。实际应用中为了避免噪声的影响，可以选取多次判断，比如连续判断两次短训练序列的长度均大于0.75 才认为分组检测成功。

IEEE802.11a 系统中的符号同步通常是在分组同步之后进行，当分组估计提供数据分组起始界限后，因为频偏会影响符号同步，先进行粗频偏估计，然后再进行符号同步，符号同步估计算法将该估计精确到量化等级。符号同步能够找到长训练序列的起始点，符号同步的方法是通过计算接收信号 $r(n)$ 和本地长训练序列参考信号 $t(n)$ 之间的互相关性实现的。如下：

$$t_s = \max \left| \sum_{n=0}^{L-1} \sum_{n=0}^{N-1} r[n+k]t^*[k] \right|$$

第4章　802.11a OFDM 基带接收端载波频率同步算法设计与验证

OFDM 技术是同时在多个重叠子信道上传输信号的，为了正确接收，必须严格保证子载波之间的正交性，但是由于多普勒频移和收发晶振的不完全相同，往往存在一定的载波频率偏差，这将破坏子载波之间的正交性，且这种频差对相位的影响还具有累加性。因此，为了保证OFDM 性能，必须进行载波频率同步。在802.11a 中前导允许接收机在实际信息分组部分尚未到达之前，采用最大似然算法估计频率偏移并进行修正。因此，载波同步是利用训练信息完成的。对于802.11a，不管是长训练符号还是短训练符号均满足这种需求。

4.1 实验目的

学习IEEE802.11a 的接收端频偏估计算法，使用Matlab 编程语言实现IEEE802.11a基带接收端得的频偏估计算法并验证。通过理论研究，频偏模拟以及实际频偏估计等实验结果的对比，深刻理解频偏的影响以及OFDM 系统的频偏敏感特性。

4.2 实验条件

PLUTO-SDR 平台一套，Matlab软件环境，WiFi 频段天线。

4.3 实验要求

使用Matlab搭建IEEE802.11a基带发送端，首先验证OFDM系统的理论抗频偏特性，其次通过模拟频偏来验证理论数据的极限值；最后通过PLUTO-SDR将基带信号经过射频发射出去并使用另一PLUTO-SDR（或者一台PLUTO-SDR自发自收）进行接收，进行频偏估计，将实际频偏估计值与理论值比较，验证算法的正确性。

4.4 实验原理

IEEE 802.11la WLAN系统的频率同步估计可以从时域上进行估计，也可以从频域上进行估计，这种方法要求训练序列至少有两个连续重复的符号，这一点在IEEE802.11la WLAN系统中其前导部分无论是长、短训练序列都可以满足要求。

在本系统中，我们从时域进行频偏估计，先用短训练序列进行粗频偏估计校正，再用长训练序列进行细频偏估计校正，两种频偏估计的方法类似，这里我们只介绍粗频偏估计。

原理：设$s[m,n]$表示发送端IFFT变换之后第m个OFDM符号的第n个子载波上的采样值，$r[m,n]$表示接收端FFT变换之前第m个OFDM符号的第n个子载波上的采样值，N为两个连续符号相同采样点之间的延时，T_S为采样时间间隔，f_1为发送载波频率，f_2为接收载波频率。

$$
\begin{aligned}
r[m,n] &= s[m,n]e^{j2\pi f_1(mN+n)T_s}e^{-j2\pi f_2(mN+n)T_s} \\
&= s[m,n]e^{j2\pi(f_1-f_2)(mN+n)T_s} \\
&= s[m,n]e^{j2\pi\Delta f(mN+n)T_s}
\end{aligned}
$$

其中 $\Delta f = f_1 - f_2$ 是发送和接收载波之间的频率频偏，频率同步估计f 推导如下，考虑两个连续的OFDM 接收符号，设中间变量z：

$$
z = \sum_{n=1}^{N} r[m,n]r^*[m+1,n]
$$

将公式代入：

$$
\begin{aligned}
z &= \sum_{n=1}^{N} s[m,n]e^{j2\pi\Delta f(mN+n)T_s}\left(s[m+1,n]e^{j2\pi\Delta f((m+1)N+n)T_s}\right)^* \\
&= \sum_{n=1}^{N} s[m,n]s^*[m+1,n]e^{j2\pi\Delta f(mN+n)T_s}e^{-j2\pi\Delta f((m+1)N+n)T_s} \\
&= e^{j2\pi\Delta fNT_s}\sum_{n=1}^{N} s[m,n]s^*[m+1,n] \\
&= e^{j2\pi\Delta fNT_s}\sum_{n=1}^{N}\left|s[m,n]\right|^2
\end{aligned}
$$

得到频偏估计为：

$$
\Delta f = \frac{\text{angle}\left(\dfrac{\sum_{n=1}^{N} r[m,n]r^*[m+1,n]}{\sum_{n=1}^{N}\left|s[m,n]\right|^2}\right)}{2\pi NT_s}
$$

其中，angle( ) angle（）表示求复数的相角运算。

IEEE802.11a系统中，短训练序列符号取样时间为50ns，延时N=16，则可以估计的最大频率误差 $\Delta f_{\max} = 1/(2NT_s) = 625\text{kHz}$，如果使用长训练序列，那么它和短训练序列的唯一区别就是N=64，则长训练序列可以估计的最大频率误差为156.25kHz。虽然用短训练序列进行频偏估计误差比较大，但是其可以估计的范围比较大，这就是我们为什么要先用短训练序列进行粗频偏估计再用长训练序列进行细频偏估计的原因。

4.5 参考设计

此系统的频偏估计分为粗频偏估计与精频偏估计，分别利用了短训练序列和长训练序列，所以理论抗频偏性能分别为20MHz/32=625KHz 和20MHz/128=156KHz；但是两种频偏估计的算法一致，只是因为训练序列的长度不同而导致频偏估计的范围不同，这里先使用粗频偏

估计估算出大范围的频率偏差，然后用精频偏估计进行小范围的频率偏差的估计。图4-1为两种频偏估计所采用的算法框架图：

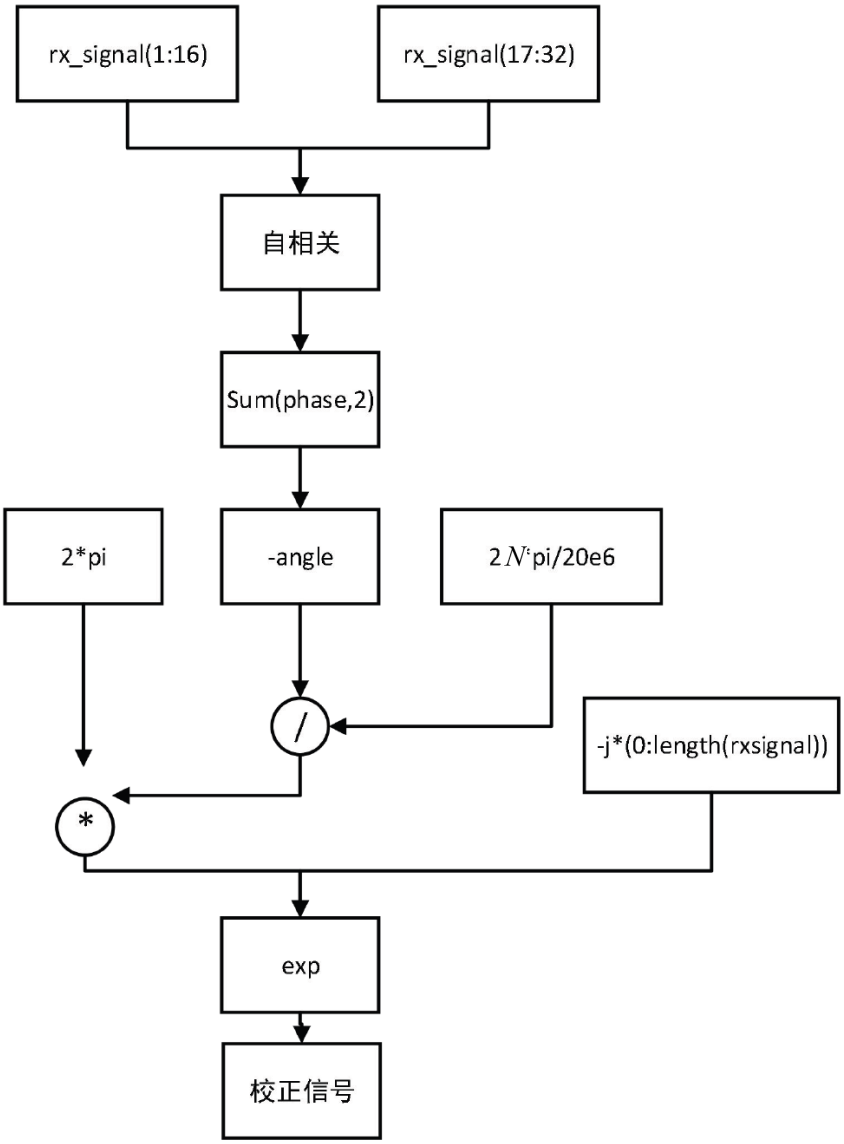


图4-1 频偏估计算法框图

第5章　802.11a OFDM 基带接收端信道估计算法设计与验证

OFDM将高速的数据分配到多个子载波上并行传输，虽然这种慢速的并行传输和循环前缀的引入，在很大程度上减小了宽带无线通信中多径信道下产生的畸变带来的影响，但仍会对子载波上的信号产生相位旋转和幅度的畸变，造成I路和Q路信号分量相互干扰等，为了在接收端恢复发送数据，必须考虑每个子载波信道所引入的幅度和相位影响。

5.1 实验目的

学习IEEE802.11a的接收端信道估计算法，使用matlab编程语言实现IEEE802.11a 基带接收端的信道估计算法并验证。通过理论研究，多径模拟以及实际信道估计等实验结果的对比，深刻理解多径的影响以及信道估计与均衡的重要性。

5.2 实验条件

PLUTO-SDR 平台一套，Matlab 软件环境，WiFi 频段天线。

5.3 实验要求

实验要求进行理论数据、模拟数据与实际数据进行对比来验证理论数据的正确性，以及此系统采用的信道估计算法的性能。通过此次实验加深对信道估计算法的理解。

5.4 实验原理

信道估计的目的就是估计出信道的时域或者频域的响应，对接收到的数据进行校正和恢复，以获得相干检测的性能增益。信道的最大多径时延，用户的移动速度，接收机的噪声都是影响信道估计算法性能的重要因素。对于OFDM 系统，信道估计是一种简便高效的方法就是估算信道的频率响应 $\hat{H}_k$ ，即对每一个子载波信道频率响应进行估算。

所谓信道估计的时域方法就是在训练符号的DFT之前，即在时域完成信道的估算。以IEEE802.11a系统为例子，长训练符号期间接收到的时域信号为

$$r_{l,n} = h \otimes x_{l,n} + w_{l,n}$$

其中，$x_{l,n}$ 是训练序列，$r_{l,n}$ 是接收到的训练序列，h为信道冲击响应，$w_{l,n}$ 为信道噪声，$\otimes$ 表示循环卷积。而在频域而言，时域的循环卷积运算在频域是较为简单的相乘运算，所以在频域表示为

$$R_{l,k} = H_k \times X_{l,k} + W_{l,k}$$

则信道的频率响应估计值为

$$\hat{H}_k = \frac{R_{l,k}}{X_{l,k}} + \frac{W_{l,k}}{X_{l,k}}$$

5.5 参考设计

参考例程采用长训练序列完成子载波信道频率响应的简单有效的估计。两个长训练符号的内容是一样的，因此对二者取平均可以改进信道估算的质量。我们知道，接收信号等于发送信号和信道频域响应的乘积再加上噪声，因此对于长训练序列有：

$$R_k = H_k \times X_k + W_k$$

其中 R_k 是接收到的两个长训练序列第k个子载波上数据的均值， H_k 是信道频域响应，W_k 是噪声信号。 X_k 是长训练序列的频域符号

LongTrain={ 1,1,-1,-1,1,1,-1,1,-1,1,1,1,1,1,1,-1,-1,1,1,-1,1,-1,1,…

1,1,1,1,-1,-1,1,1,-1,1,-1,1,-1,-1,-1,-1,-1,1,1,1,-1,-1,1,-1,1,-1,1,1}

根据如上算法，参考例程中算法实现的框图如图5-1：

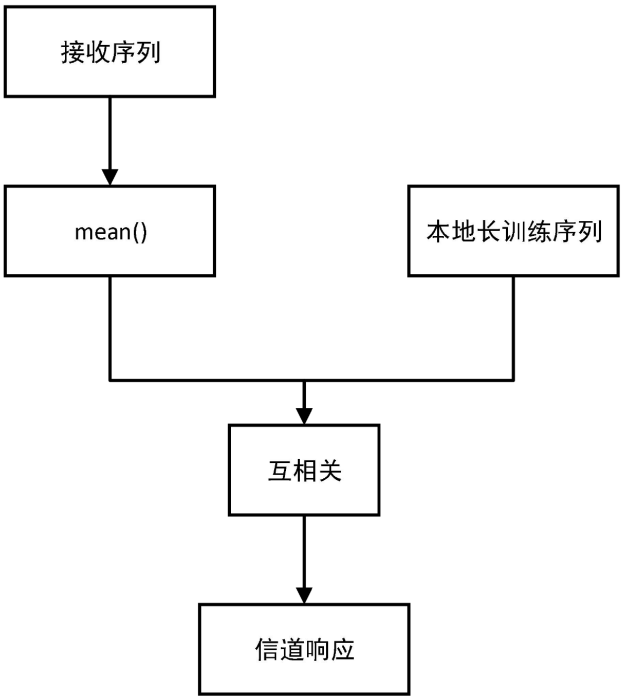


图5-1 信道估计实现框图

第6章　802.11a OFDM 基带接收端系统搭建

接收端的OFDM基带处理器设计与发端相比更为复杂，因为它除了要完成发射端各种处理的反向功能之外，更为重要的是要实现对接收信号的同步处理，这在整个通信系统中占据重要的地位。而OFDM系统又存在对频率偏差敏感的缺点，因此如果不能很好的解决同步的问题，会给系统性能带来非常严重的地板效应，既无论怎样增加信号的发射功率，都不能显著的改善系统性能。

对一个通信系统而言，必须要考虑如下一些同步问题，如接收机不知道符号的定时起始位置、发射机与接收机的载波频率存在偏差、发射机与接收机的采样频率不可能完全相同等。因此，在OFDM系统中主要由以下两个同步方面的要求：首先是时间同步，包括帧的检测和符号同步，以保证正确检测到新数据的到达，并保证IFFT和FFT起止时刻一致；其次是载波同步，要消除接收机的本振频率与发射机的本振频率和相位的偏差引起的系统性能降低；除了同步问题外，频率选择性信道的均衡也是一个关系到系统性能的重要性问题，通过本讲实验建立完整的IEEE802.11a接收端，验证整个通信系统的性能。

6.1 实验目的

IEEE802.11a接收端的设计更为复杂，因为它除了要完成发射端各种处理的反向功能之外还要增加同步等处理模块，具体的处理模块算法在上面的章节中已经详细讲过，在这里不再详述，通过此次实验整体掌握接收端的流程以及各个模块的衔接，最后通过结果信息验证算法与PLUTO-SDR性能。

6.2 实验条件

PLUTO-SDR平台一套，Matlab软件环境，WiFi频段天线。

6.3 实验要求

在此系统中，利用训练序列分别实现了帧同步检测和定时符号同步，然后利用FFT算法分离长训练序列，并利用长训练序列在频域实现信道估计与均衡；最后分别根据信号域与数据域有序的进行解映射、解交织、解打孔、译码、解扰、解CRC等算法操作。由于信号域部分在发端并没有加扰，所以不需要进行解扰操作。当信号域的信息被正确解出来后，根据信号域的码率、数据长度以及调制方式等信息对数据域的数据进行最后的解调，并在每个不同的操作步骤后打印出必要的信息，最后绘出功率谱密度、星座图等信息。

6.4 实验原理

IEEE802.11a接收端比发送端复杂很多，除了对发送端的逆向之外，还增加了同步模块。其中同步包括定时同步（分为分组检测和符号同步两部分），频率同步，采样同步。

同步问题来自:

a. 发收端晶振的本振点偏差导致频偏

b. 多普勒频移（时间选择性问题）导致频偏

c. 多径（频率选择性问题也即是时延扩展）

因此，关于接收端的程序框架我们设计如下：首先是同步模块，然后是发送端逆过程模块，如图6-1：

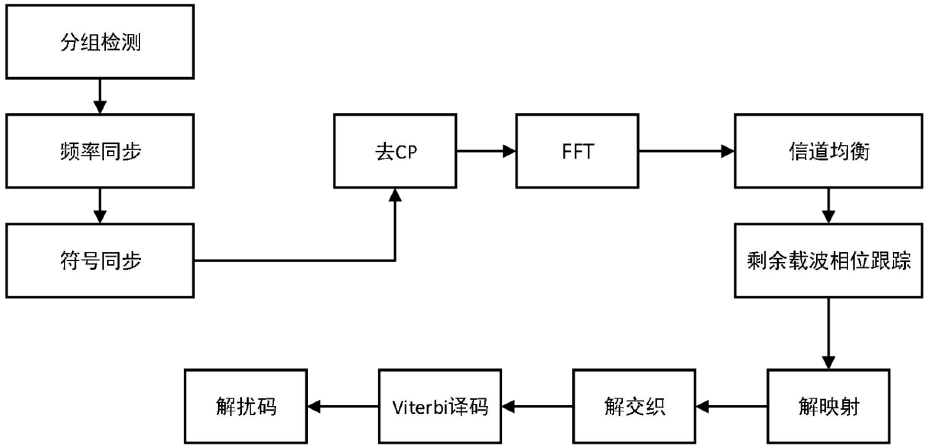


图6-1 IEEE802.11a 接收端系统框图

定时同步分为分组检测和符号同步两部分，分组检测是粗同步，检测每一帧何时到达；符号同步是细同步判断每一个OFDM符号的起始位置。频率同步分为频偏估计，信道均衡，剩余载波相位跟踪。其中频偏估计主要是解决晶振的本振点偏差和多普勒频移问题，信道均衡解决多径问题，剩余载波相位跟踪解决星座图的旋转问题。设置接收端采样时钟40MHz，IEEE802.11a协议规定采样率20MHz，采样同步在参考例程中简化为隔点采样的简单方法，读者可以采用更好的采样同步算法增加系统性能。

6.5 参考设计

接收端包括若干子模块，其中关键模块的算法已经在之前的实验中阐述，这里只做概念性的讲解，如有疑问，请参考上文实验或其他教材。

在同步模块，实现流程如下：信号经过A/D采样之后，通过分组检测模块来进行粗定时，确定帧的起始位置，然后进行粗频偏估计校正模块和精频偏估计校正模块，然后利用训练符

号进行符号定时，找出OFDM符号的起始点，接着经过去循环前缀模块后进入FFT模块，从此由时域到达频域；利用频域的长训练序列进行信道估计，然后进行剩余载波相位跟踪，之后基本就是发端的逆过程和逆序。

IEEE 802.11a WLAN系统的频率同步估计可以从时域上进行估计也可以从频域上进行估计，这种方法要求训练序列至少有两个连续重复的符号，这一点在IEEE 802.11a WLAN系统中其前导部分无论是长、短训练序列都可以满足要求。

在本系统中，采用时域频偏估计，先利用短训练序列进行粗频偏估计校正，再利用长训练序列进行细频偏估计校正，两种频偏估计的方法相同，这里我们只介绍粗频偏估计。因为频偏会影响符号同步，所以在做完分组检测和频偏估计之后再进行符号同步，这样结果比较准确。本系统中，符号同步能够找到长训练序列的起始点。符号同步的方法是通过计算接收信号 R_n 和本地长训练序列参考信号 T_n 之间的互相关实现。

从面的分析可以看出，频偏估计只采用训练序列并非完全准确，如果一帧数据中包括多个OFDM符号，那么随着符号的增加每个符号都会有一定的误差，由此带来的问题是星座图的旋转。此影响要求接收机在接收数据符号时必须对载波相位进行跟踪。在IEEE802.11a发射端每个OFDM 符号中插入了四个导频子载波，所以在接收端利用导频对每个OFDM 符号载波相位辅助跟踪是最简单的方法（有些文件中利用导频的载波跟踪合并入了信道估计中）。IEEE 802.11a包括了四个发送数据中预先确定的子载波，这些专用的子载波指的就是导频载波，导频的主要目的在于帮助接收机准确的跟踪载波。

在第n个接收符号的DFT之后，导频子载波 $R_{n,k}$ 等于信道频率响应 H_k 和已知导频符号 $P_{n,k}$ 的乘积根据余下的频率误差旋转以后的结果:

$$R_{n,k} = H_k P_{n,k} e^{j2\pi\Delta f}$$

所以相位估算为：

$$\tilde{\varphi}_n = \text{angle}\left(\sum_{k\in\Omega_p} R_{n,k} (H_k P_{n,k})^* \right)$$

这里，Ω_p 表示导频集合。

这样便得到了每个OFDM符号的相位跟踪的估计值，然后将每个OFDM 符号里的数据（48 个有用数据）进行相位纠正，也就是将这48个数据都乘以同一个纠正相位值。不同OFDM符号得到的纠正相位值不同。

在完成接收端的同步模块之后，接下来便是对发送端各个模块进行一系列的逆过程了。

首先将接收数据分组，每组48个样值。第一组是SIGNAL域，进行BPSK解调，解交织，1/2码率viterbi译码，得到SIGNAL字段的数据信息，然后可以得到发送端参数LENGTH和RATE，根据这两个参数可以知道data字段的相关参数包括调制方式，编码方式等等，然后就可以进行跟发送端相反的data字段生成的逆过程：解调制，解交织，Viterbi 译码，解扰，最后得到有用的信息数据。

由于接收端的逆过程模块大部分都是根据发送端的生成过程设计实现的，只要参照前面发送端的生成过程，就可以很简单的完成逆过程这一部分。这里我们只介绍维特比解码。

在IEEE802.11a协议里，关于译码的实现方法，并没有特殊建议。参考例程中采用了较简单的实现方案硬判决解调和Viterbi译码。硬判决解调的实现，正如我们所知，解调时的反映射，就是把复数符号反映射到星座图的相应位置，根据一定的判决法则。解调输出0，1比特信息。在本文中使用的反映射判决度量为欧氏距离。通过计算接收到的复数符号和信号集里复数符号之间的欧氏距离，来决定判决结果。欧氏距离越大则相似程度越大，最相似的将作为反映射输出。

Viterbi解码算法：卷积码的译码和编码一样，也是利用码字前后的相关性，在网格图中找出一条最相似于发送码字的连续的路径。目前被广泛使用的Viterbi解码算法 (AJ. Viterbi于1967年提出)是一种最大似然译码。它的思路是：以断续的接收码流为基础，逐个计算它与其他所有可能出现的、连续的网格图路径的距离，选出距离最小者作为译码估值的输出。假设发送端发送的码字序列是 $\mathbf{S}=[S_0,S_1,\cdots,S_i,\cdots]$，接收端收到的码字序列是 $\mathbf{R}=[R_0,R_1,\cdots,R_i,\cdots]$。由于存在噪声干扰，使得 $\mathbf{R}$ 不等于 $\mathbf{S}$，在接收端，用所有可能码字序列 $\mathbf{S}_j=[S_{j,0},S_{j,1},\cdots,S_{j,i},\cdots]$ 与接收码字序列 $\mathbf{R}$ 进行比较。最大似然译码就是把似然度最大的那个序列作为译码估值序列输出。译码准则是：

$$\tilde{\mathbf{S}}=max\left(\sum_{l=1}^{L}\log[\Pr(\mathbf{R}^l/\mathbf{S}_j^l)]\right)$$

硬判决Viterbi译码：硬判决Viterbi译码算法是，在所有候选序列 $\mathbf{S}_j$ 中，寻找与接收序列 $\mathbf{R}$ 汉明距离最小的那个作为译码估值序列 $\mathbf{S}$。此时最大似然度就等效为最小汉明距离：

$$\max\left(\log\Pr\left[\frac{\mathbf{R}}{\mathbf{S}_j}\right]\right)\Leftrightarrow\min\left[d_j\right]$$

第 7 章 软件无线电平台 ADALM-PLUTO 简介

就像冥王星（天体）是一颗矮星一样（类似于一颗小行星，但缺乏对其进行分类所需的某些技术标准），PLUTO-SDR 是一个类似于软件定义无线电的主动学习模块，但它缺乏对其成为软件定义无线电的一些性能/技术标准。

虽然 PLUTO-SDR 是通信原理课程设计第一次体验软件定义无线电（Software Defined Radio，SDR）的最佳学习工具，但它并不意味着取代或替代现有的各种专业软件定义无线电。PLUTO-SDR 旨在为使用者提供射频 SDR 功能，因此，每个用户都应该意识到 PLUTO-SDR 的局限性。



(a) 外观　　(b) 内部功能模块

图 7-1 PLUTO-SDR 的外观及内部功能模块

图 7-1 给出了 PLUTO-SDR 的外观及内部功能模块示意图。ADALM-PLUTO 内部的无线电芯片是 AD9363，这是一种基于直接转换接收机的高性能、高度集成的射频敏捷收发器。接收子系统包括低噪声放大器（LNA）、直接转换混频器、可配置模拟滤波器、高速模数转换器（ADC）、数字抽取滤波器和 128 抽头有限脉冲响应（FIR）滤波器，以适当的采样率产生 12 位输出信号。接收链增加了可配置的自动增益控制（AGC）或手动增益控制模式、直流偏移校正和正交校正。产生的接收到的 I 和 Q 信号被传递到数字基带处理器，相应处理器为 Xilinx Zynq SoC。

PLUTO-SDR 的发送子系统也使用直接转换架构。它接收来自基带处理器 Xilinx Zynq SoC 的 12 位 I 路和 Q 路样本，通过 128 抽头有限脉冲响应（FIR）滤波器、数字插值滤波器、

高速数模转换器（DAC）、模拟滤波器、直接转换混频器，并通过一个小功率放大器（PA）输出到天线。

AD9363 内部的全集成锁相环（PLL）为接收和发送通道提供时钟和本地振荡器，并为 ADC、DAC 和输出采样率提供时钟。

Xilinx Zynq 全可编程 SoC（AP SoC）将基于 ARM 处理器的软件可编程性与 FPGA 的硬件可编程性集成在一起，实现硬件加速，同时将 CPU、DSP、ASSP(Application Specific Standard Parts,专用标准产品)和混合信号功能集成到单个设备中。此类设备具有单核 ARM Cortex™-A9 处理器与基于 28nm Artix®-7 的可编程逻辑相匹配，配备常用的外围设备（如，USB、SPI 等）。

7.1 PLUTO-SDR 的参数及功能特点

参数规格

电源：

直流输入(USB)：4.5V-5.5V

转换性能和时钟：

ADC 和 DAC 采样率：65.2kSPS 到 61.44MSPS

ADC 和 DAC 分辨率：12 位

频率精度：±25ppm

射频性能：

调谐范围：325MHz 到 3800MHz

Tx 输出功率：7dBm

Rx 噪声系数：<3.5dB

Rx 和 Tx 调制精度(EVM)：–34dB

射频屏蔽：无

数字规格

USB：2.0 On-The-Go

内核：双 ARM Cortex®-A9，频率为 667 MHz

FPGA 逻辑单元：28k

DSP Slice：80

DDR3L：4Gb (512MB)

QSPI 闪存：256Mb (32MB)

特点

独立的收发信道；

直观的图形用户界面；

多操作系统：Windows、Linux；

提供许多适合 SDR 项目的可用在线教程；

开源。

功能

利用实时射频信号进行空中实验和现场测试；

自定义无线功能的快速原型；

通过动手实践，学习无线通信的概念和设计技巧。

7.2 PLUTO-SDR 软件配置

图 7-2 给出了顶层 Windows 或者 Linux 操作系统下 MATLAB/Simulink 开发软件与 PLUTO-SDR 的数据交换方式。二者的数据交换采用了服务器/客户端的通信方式，其核心是 libiio 库。libiio 库是硬件低层细节的抽象，提供了简单但完整的编程接口，可用于绑定各种语言（C、C++、C#、Python）的高级项目。

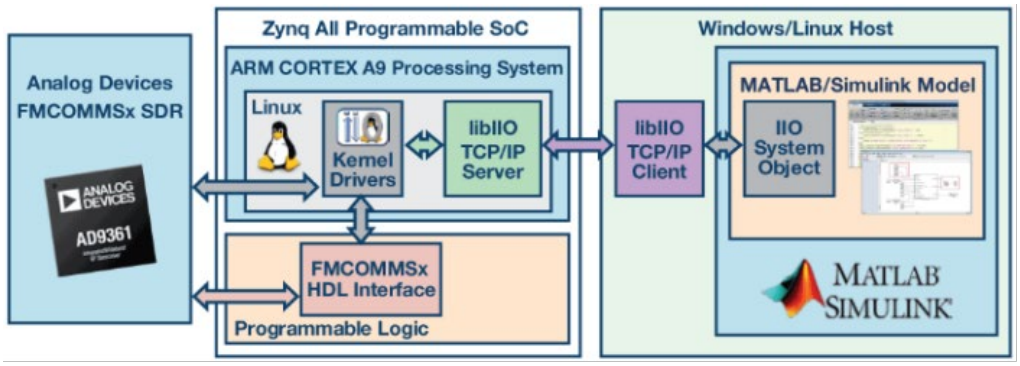


图 7-2 MATLAB/Simulink 开发软件与 PLUTO-SDR 的数据交换方式

利用 PLUTO-SDR 进行软件无线电程序开发，需要借助于 Windows 或 Linux 操作系统下的软件开发环境，最常用的开发软件为 MATLAB/Simulink 等。首先在 MATLAB 开发环境下编写通信系统的基带信号收、发程序；之后，将待发送的基带信号数据通过 libiio 库函数发送给 PLUTO-SDR 系统，由 PLUTO-SDR 完成通带信号发送过程；另一方面，PLUTO-SDR 的接收单元将接收到的通带信号下变频至基带，并通过 libiio 库函数交给上位机的 MATLAB 中的接收软件。

Analog Devices 开发的 libiio 库，显著简化了用于连接 Linux Industrial I/O (IIO) 器件（例如 Pluto 上的 AD9363）的软件的开发工作。开源(GNU Lesser General Public License V2.1) 库抽象出硬件的低层细节，并提供一个可用于高级项目的简单但完整的编程接口。

可以使用 libiio 在项目原型设计阶段与 Pluto SDR 进行接口连接，以便向或从 MATLAB/Simulink 或 GNURadio 等工具的模型发送或接收样本数据流，具体过程如图 7-3 所示。

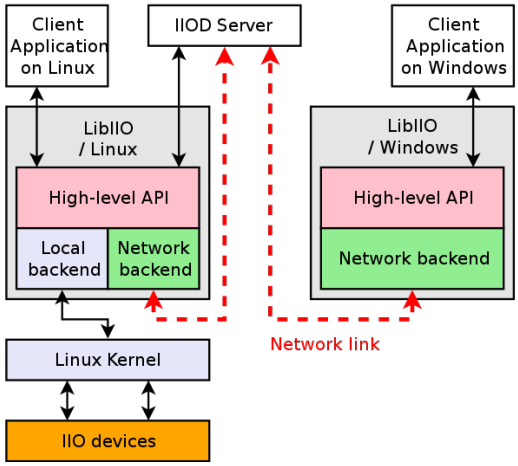


图 7-3 基于 libiio 的发送/接收样本数据流

7.3 IIO Oscilloscope

IIO Scope 是 ADI 公司开发的一款开源 Linux 应用程序，运行在 Xilinx Zynq 中 ARM Cortex-A9 的内核上，能够显示 PLUTI-SDR 设备所获取的实时数据。这些数据可以在时域中、频域中或以星座图的形式显示。支持加载各种预先存储的基带调制信号（如 QAM、LTE 等）循环发射。IIO Scope 提供一个图形用户界面，用于更改或读取 SDR 设备的配置。

点击桌面 IIO Oscilloscope 图标：



图 7-4 IIO Oscilloscope 图标

软件首次运行会弹出如图 7-5 左侧界面，手动选择点击“USB Device”，同时点击“刷新”按钮，出现如下右图所示，在 Contex Info 和 IIO Devices 一栏可看到相关的设备信息，代表一切正常。

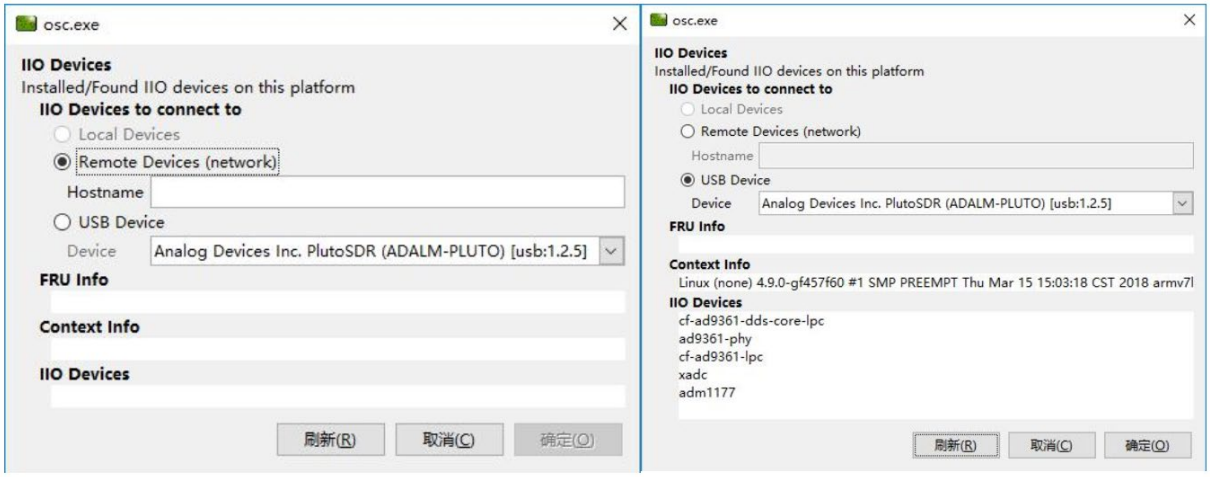


图 7-5 IIO Oscilloscope 初次运行界面

点击“确定”，进入如图 7-6 所示的参数设置界面：

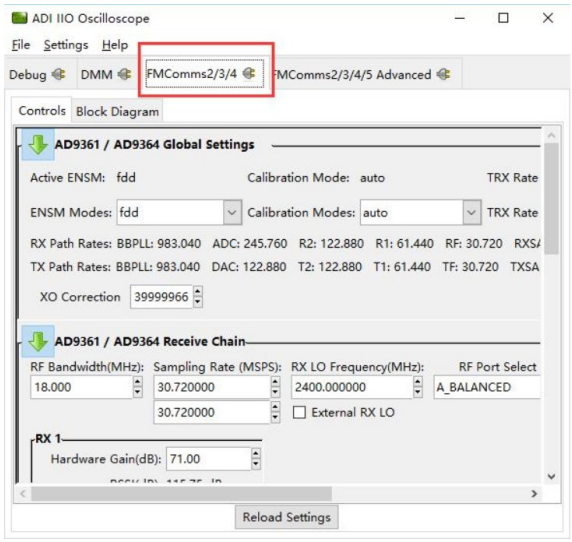


图 7-6 IIO Oscilloscope 参数设置界面

点击“FMComms2/3/4”一项，其他的 PLUTO_SDR 选项用不到，不用管。在“FMComms2/3/4”一项中，主要由以下几部分组成：

1. AD9361/AD9364 Global Settings

芯片的全局配置。在此主要对 PLUTO_SDR 的 AD9363 进行设置，默认即可。

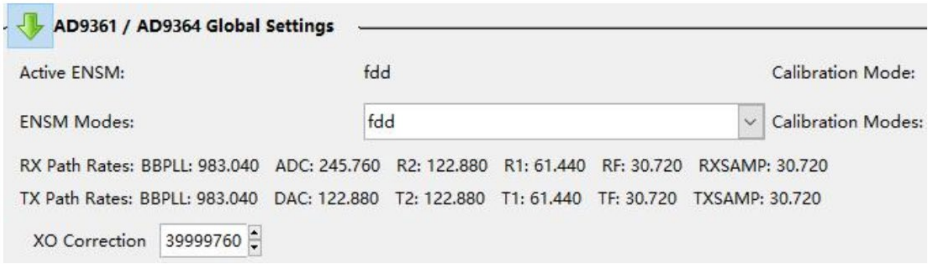


图 7-7 AD9361/AD9364 全局设置界面

2. AD9361/AD9364 Receive Chain

接收通道射频配置。主要有 RF Bandwidth 射频带宽、Sampling Rate 射频采样率、RX LO Frequency 射频频点等参数，具体参数范围可参考 PLUTO_SDR 的射频指标；RF Port Select 默认选择 A_BALANCED；Tracking 将三项都选择上；在 Gain Control Modes 一项中，可以选择 manual 手动增益模式、slow_attack 慢自动增益控制和 fast_attack 快自动增益控制模式。

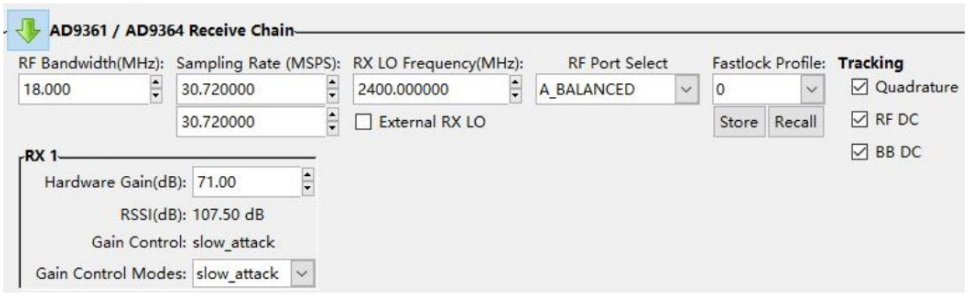


图 7-8 AD9361/AD9364 接收通道射频配置界面

3. AD9361/AD9364 Transmit Chain

发送通道射频配置。主要有 RF Bandwidth 射频带宽、Sampling Rate 射频采样率、TX LO Frequency 射频频点等参数，具体参数范围可参考 PLUTO_SDR 的射频指标；RF Port Select 默认选择 A；Attenuation 设置发送通道的衰减值（dB），根据实际情况来设定，设定标准是衰减不能太大否则接收不到信号，也不能太小造成接收饱和失真。在 SMA 直连的情况下默认为 20dB。

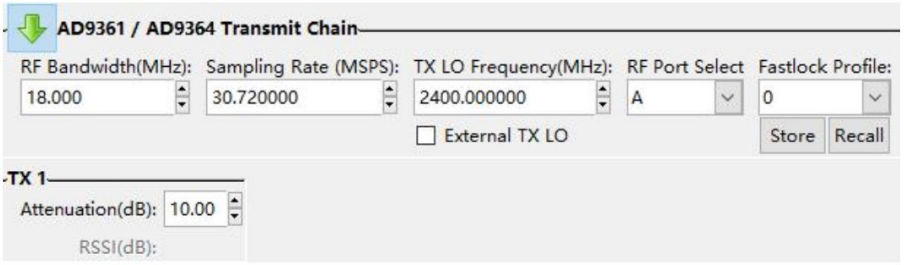


图 7-9 AD9361/AD9364 发送通道射频配置界面

4. FPGA Settings

数据源选择配置模块。可选择单音信号 One CW Tone、双音信号 Two CW Tone 等。一个适当的配置参数如图 7-10 所示：

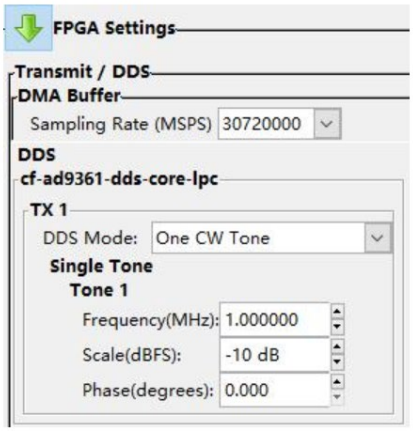


图 7-10 FPGA 设置界面

在 ADI IIO Oscilloscope-Capture1 对话框中手动勾选 voltage0 和 voltage1，如图 7-11 所示。

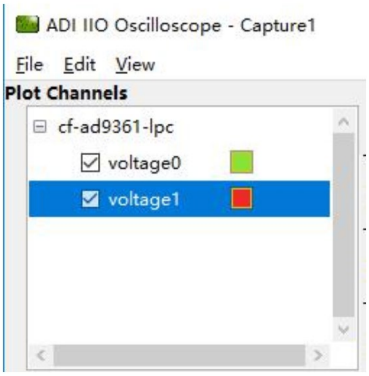
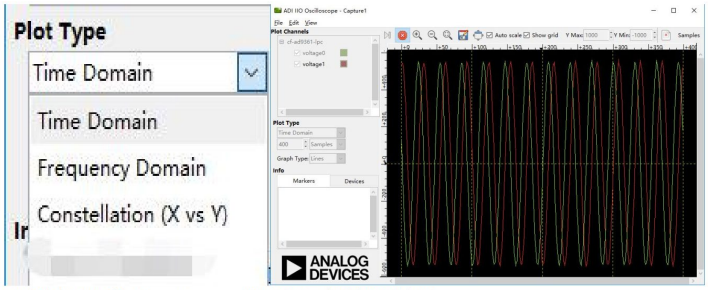


图 7-11 ADI IIO Oscilloscope 观测参数设置界面

点击运行按钮，会实时显示接收到的时域信号（分别对应 I 路和 Q 路），如图 7-12 所示：



通过选择不同的 Plote Type，可分别显示频谱和星座图：

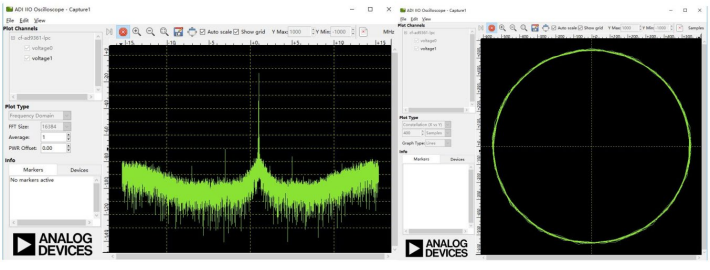


图 7-12 IIO Oscilloscope 波形显示

7.4 MATLAB 环境配置

通过上述分析可知，PLUTO-SDR 通过 libiio 向上层控制机（MATLAB）提供了简单易用的设备配置和数据通道接口，通过通用的代码配置格式，可以方便的对下层硬件进行快速配置并可实时修改。数据交换过程采用了 TCP/IP 网络协议以确保数据的可靠传输。

PLUTO-SDR 可以支持 3 种类型的工作模式，（1）单工，只作为发射机或者接收机；（2）半双工，发送通道作为信号源，实时接收数据；（3）全双工，实时发射和接收数据。

MATLAB 环境下 PLUTO-SDR 的配置文件为 ad9361.cfg，其作用是将系统对象接口链接到 IIO 数据通道或 IIO 属性。ad9361.cfg 文件内容如下所示：

```
data_in_device = cf-ad9361-dds-core-lpc
data_out_device = cf-ad9361-lpc
ctrl_device = ad9361-phy
channel = RX_LO_FREQ,IN,out_altvoltage0_RX_LO_frequency,
channel = RX_SAMPLING_FREQ,IN,in_voltage_sampling_frequency,
channel = RX_RF_BANDWIDTH,IN,in_voltage_rf_bandwidth,
channel = RX1_GAIN_MODE,IN,in_voltage0_gain_control_mode,
channel = RX1_GAIN,IN,in_voltage0_hardwaregain,
channel = RX1_RSSI,OUT,in_voltage0_rssi,
channel = TX_LO_FREQ,IN,out_altvoltage1_TX_LO_frequency,
channel = TX_SAMPLING_FREQ,IN,out_voltage_sampling_frequency,
channel = TX_RF_BANDWIDTH,IN,out_voltage_rf_bandwidth,
```

上述代码中，data_in_device 表示用于向设备发送数据的 linux 驱动程序的名称；data_out_device 表示用于从设备读取数据的 linux 驱动程序的名称；ctrl_device 表示用于控制和监视设备的 linux 驱动程序的名称；channel 表示通道配置，其格式定义为<通道名称，通道类型，Linux 属性>。

在用户的 MATLAB 代码中需要构建 IIO 系统对象，设置 IP 地址，定义目标设备名称，设置输入、输出通道的大小和采样点数。具体代码如下：

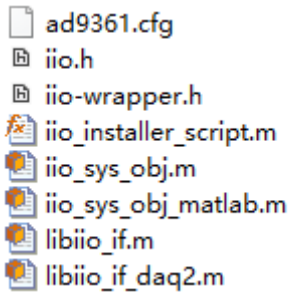
```
ip = '192.168.2.1';
txdata = xxx_tx_func;   %%调用基带信号发送程序，得到发送的基带信号数据
txdata = round(txdata .* 2^14);     %% 调整发送数据的取值范围
txdata=repmat(txdata, 8,1);            %% 实现多次发送
%% Transmit and Receive using MATLAB libiio
% System Object Configuration
s = iio_sys_obj_matlab; %   构建 IIO 系统对象
```

```
s.ip_address = ip;       %设置 IP 地址
s.dev_name = 'ad9361';   %定义目标设备名称
s.in_ch_no = 2;     %设置输入、输出通道
s.out_ch_no = 2;
s.in_ch_size = length(txdata);   %设置输入、输出通道的采样点数
s.out_ch_size = length(txdata).*2;
s = s.setupImpl();
input = cell(1, s.in_ch_no + length(s.iio_dev_cfg.cfg_ch));
output = cell(1, s.out_ch_no + length(s.iio_dev_cfg.mon_ch));
```

此外，还需要利用 IIO 系统对象设置 AD9363 属性：

```
% Set the attributes of AD9361
input{s.getInChannel('RX_LO_FREQ')} = 2e9;     %%接收本振频率
input{s.getInChannel('RX_SAMPLING_FREQ')} = 40e6;   %%接收采样频率
input{s.getInChannel('RX_RF_BANDWIDTH')} = 20e6;     %%接收带宽
input{s.getInChannel('RX1_GAIN_MODE')} = 'manual';   %% slow_attack manual
input{s.getInChannel('RX1_GAIN')} = 10;   %% 接收增益
% input{s.getInChannel('RX2_GAIN_MODE')} = 'slow_attack';
% input{s.getInChannel('RX2_GAIN')} = 0;
input{s.getInChannel('TX_LO_FREQ')} = 2e9+40e3;     %% 发送本振频率
input{s.getInChannel('TX_SAMPLING_FREQ')} = 40e6;   %% 发送采样频率
input{s.getInChannel('TX_RF_BANDWIDTH')} = 20e6;     %% 发送带宽
```

注意，所需的任何例程必须包含以下配置文件：



作为示例说明，下面给出 OFDM 通信系统的射频参数设置代码：

```
clearvars -except times;close all;warning off;
pluto_init.ip = '192.168.2.1';
pluto_init.rx_freq = 2000e6;     %% 接收端载波频率
pluto_init.rx_samp = 40e6;       %% 接收端采样频率
pluto_init.rx_bw = 20e6;         %% 接收端带宽
```

```
pluto_init.rx_gain_mode = 'manual';
pluto_init.tx_freq = 2000e6;        %% 发送端载波频率
pluto_init.tx_samp = 40e6;          %% 发送端采样频率
pluto_init.tx_bw = 20e6;            %% 发送端带宽
```

PLUTO-SDR配置函数示例如下：

```
function [ input_content, output_content, s ] = pluto_config( pluto_init, txdata )
% System Object Configuration
s = iio_sys_obj_matlab; % MATLAB libiio Constructor
s.ip_address = pluto_init.ip;
s.dev_name = 'ad9361';
s.in_ch_no = 2;
s.out_ch_no = 2;
s.in_ch_size = length(txdata);
s.out_ch_size = length(txdata)*24;
s = s.setupImpl();
input_content = cell(1, s.in_ch_no + length(s.iio_dev_cfg.cfg_ch));
output_content = cell(1, s.out_ch_no + length(s.iio_dev_cfg.mon_ch));

input_content{s.getInChannel('RX_LO_FREQ')} = pluto_init.rx_freq;
input_content{s.getInChannel('RX_SAMPLING_FREQ')} = pluto_init.rx_samp;
input_content{s.getInChannel('RX_RF_BANDWIDTH')} = pluto_init.rx_bw;
input_content{s.getInChannel('RX1_GAIN_MODE')} = pluto_init.rx_gain_mode;
input_content{s.getInChannel('RX1_GAIN')} = 10;
input_content{s.getInChannel('TX_LO_FREQ')} = pluto_init.tx_freq;
input_content{s.getInChannel('TX_SAMPLING_FREQ')} = pluto_init.tx_samp;
input_content{s.getInChannel('TX_RF_BANDWIDTH')} = pluto_init.tx_bw;

txdata = int16(txdata.*2^14);
input_content{1} = real(txdata);
input_content{2} = imag(txdata);
end
```